COMP1020 – Spring 2026 Term Project Proposal

Coffee Shop POS and Loyalty System

Team VinTony

Team Introduction

Our team, VinTony, consists of 5 members with different levels of programming skills. Some members have prior experience and achieved good results in COMP1010, while others are still developing their programming fundamentals. We aim to collaborate effectively and apply OOP, Data Structures, and Algorithms into a practical system.

Members

- **Nguyen Duc Nhat** – Team Leader (prior experience in OOP)
- **Dinh Nguyen Gia Khanh** – Beginner in OOP
- **Khuc Nhat Minh** – Beginner in OOP
- **Le Duc Huy** – Beginner in OOP
- **Nguyen Duy Hieu** – Beginner in OOP

Project Description

Idea

The proposed project is a Coffee Shop POS and Loyalty System, developed as a Java desktop application. The system simulates how a café manages orders, customers, and inventory – modeling real-world entities as objects and using appropriate data structures to solve specific problems (e.g. queue for orders, HashMap for storage).

Purpose

We aim to bridge the gap between classroom theory and practical application. By building a working system around a familiar context (a coffee shop), team members can see directly how OOP design and data structures are applied together in a coherent, real-world scenario.

Objectives

- Apply OOP principles: encapsulation, inheritance, polymorphism, and abstraction
- Apply Data Structures and Algorithms appropriately in real-world scenarios
- Build a system where different components interact logically and cleanly

Product Type & Problem Solved

Desktop Program (Java GUI Application). Existing POS systems are often too complex for students to understand. Our system focuses on simplicity and clarity, demonstrating how OOP and Data Structures work together in practice.

- Helps students understand how classes interact in a real system
- Shows how data structures improve performance and organisation
- Provides a hands-on example of design patterns in context

Target Users

Café Staff

- Create and manage orders
- Add/remove toppings
- Process orders efficiently

Customers

- Identified by phone number
- Earn loyalty points
- Use vouchers for discounts

Functional Specification

OOP Design and Justification

Encapsulation
Each class manages its own data (e.g., `Customer` manages loyalty points).
Why: Ensures data integrity and prevents external modification.
Interaction: `Order` accesses customer data via controlled methods.

Inheritance
`Coffee` and `Tea` inherit from `Beverage`.
Why: Avoids code duplication.
Interaction: `Order` works with `Beverage` but supports all subclasses.

Polymorphism
Different drinks implement `calculatePrice()` differently.
Why: Enables flexible system design.
Interaction: `Order` calls `calculatePrice()` without knowing the exact type.

Abstraction
`Beverage` defines a common interface.
Why: Enforces consistent design.
Interaction: The UI interacts with `Beverage` without depending on implementation.

Design Patterns

Factory Pattern
Creates beverage objects dynamically.
Why: Avoids hardcoding object creation; easy to extend with new drink types.

Decorator Pattern
Adds toppings dynamically to drinks.
Why: Extends functionality without modifying the base class (open/closed principle).

Singleton Pattern
Used for the Inventory Manager.
Why: Ensures one global instance, preventing inconsistent inventory state.

MVC Architecture
Separates the UI, business logic, and data layers.
Why: Improves scalability; changes to one layer do not break others.

Data Structures & Algorithms

Queue (FIFO)
Used for order processing.
Why: Matches real-world workflow — first order placed is first served.

Priority Queue
Used for VIP or urgent orders.
Why: Handles priority cases ahead of regular ones.

HashMap
Used for customer and inventory storage.
Why: $O(1)$ average-case lookup for real-time performance.

Stack
Used for undo/redo operations.
Why: LIFO behaviour maps naturally to action history.

Tree
Used for the menu structure.
Why: Represents hierarchical data (categories $\rightarrow$ drinks $\rightarrow$ sizes).

Sorting
Orders sorted by time/price; customers by loyalty points.
Why: Improves readability and supports efficient search. $O(n \log n)$.

Binary Search
Used for efficient lookup in sorted lists.
Why: Reduces lookup from $O(n)$ to $O(\log n)$.

Complexity: Queue/HashMap $O(1)$; Sorting $O(n \log n)$; Tree $O(\log n)$.

Functional Features

Order Management

- Create, edit, delete orders
- Add/remove toppings
- Undo/redo actions

Customer Management

- Add, update, delete customers
- Track loyalty points

Voucher System

- Create and apply vouchers

Inventory Management

- Track ingredients
- Update quantities

System Interaction

Orders are created, processed through queues, update inventory, and interact with customer data. Design patterns ensure clean interaction between components.

Project Timeline

Ph.	Name	Description	Start	End
1	Planning & Analysis	Proposal preparation.	Mar 12	Mar 26
2	Requirements Definition	Identify system requirements, choose main features, and draw simple diagrams.	Mar 27	Apr 2
3	Design & Prototyping	Design the classes (`Beverage`, `AddOn`, `Order`, etc.) and draft GUI.	Apr 3	Apr 16
4	Implementation	Write core Java code for orders, queue, and inventory using Data Structures.	Apr 17	May 7
5	Refactoring & Feature Dev.	Fix bugs, clean up code, and add loyalty points and voucher features.	May 8	May 14
6	Testing & Deployment	Run tests with different cases, fix errors, ensure everything runs smoothly.	May 15	May 21
7	Finalization & Docs	Write the final report and prepare slides for the presentation/demo.	May 22	May 28

Term Project Team Agreement Form

Course: COMP1020
Group Number: 5
Project Title: Coffee Shop POS and Loyalty system
Team Name (if any): VinTony
Team Leader: Nguyễn Đức Nhật
Date: 16/03/2026

Team Member Information

Name	Student ID	Signature / Acknowledgment
Nguyễn Đức Nhật	V202502368	Nhật
Đinh Nguyễn Gia Khánh	V202502398	Khánh
Khúc Nhật Minh	V202502361	Minh
Lê Đức Huy	V202502747	Huy
Nguyễn Duy Hiếu	V202502909	Hiếu

Agreement Terms

By signing this agreement, we, the members of the above project team, acknowledge and agree to the following:

1. **Equal Commitment**: We are each responsible for contributing actively and fairly to the project's success.
2. **Role Assignment**: We will collaboratively determine task assignments based on individual strengths and project requirements.
3. **Communication**: We agree to communicate regularly, respond promptly to messages, and attend scheduled team meetings.
4. **Accountability**: We understand that individual contributions will be evaluated, and non-participation may negatively affect final grades.
5. **Conflict Resolution**: We will attempt to resolve conflicts internally; if unresolved, the issue will be reported to the instructor.
6. **Reporting Non-Participation**: The team leader will report any non-contributing members, and members may also report non-performing leaders.

Acknowledgment:
We understand and agree to abide by the terms outlined above. Failure to meet these expectations may result in individual penalties.